Thamill Indian B U
Cohort-15 (AI Engineer)

Day 14 Post-Session Assignment & Engineering Reflection

AI Engineering Program | Day 14: Standardized Tool Integration with Model Context Protocol (MCP)
Repository: https://github.com/ThamillIndianBlend360/AI-Assignments/tree/main/Day14

Part 1 - Output & Artifact Evidence

The existing safe HR Leave Assistant tools (get_available_leave_balance and create_leave_request) were exposed through a standardized Model Context Protocol (MCP) server using Python's FastMCP framework. The MCP server runs over standard I/O (stdio) transport and is registered in the VS Code workspace configuration (.vscode/mcp.json), allowing an MCP-capable client (VS Code / Claude extension) to perform capability discovery and tool invocation dynamically.

Repository Link & Code Artifacts

• **Repository URL:** https://github.com/ThamillIndianBlend360/AI-Assignments/tree/main/Day14
• Day14/mcp_server.py: FastMCP server instance defining tool decorators, docstring-based tool metadata, Pydantic/type-annotated parameters, deterministic balance checks, and idempotency caching.
• .vscode/mcp.json: MCP client server registration configuring stdio transport with workspace Python virtual environment.

Normal-Case Verification Evidence

Observed Client Execution Trace (VS Code MCP Client -> FastMCP Server):

```
[MCP CLIENT REQUEST]
Call tool: get_available_leave_balance
Arguments: {"user_id": "E101"}

[MCP SERVER EXECUTION]
FastMCP stdio transport processed tool call request for 'get_available_leave_balance'.

[RETURNED RESPONSE]
{
  "status": "SUCCESS",
  "user_id": "E101",
  "name": "Priya",
  "leave_balance": 3
}
```

Thamill Indian B U
Cohort-15 (AI Engineer)

Part 2 - Results & Observations

What Worked?

1. Dynamic Capability Discovery: The MCP client automatically discovered available tools, input argument types, and descriptions via the FastMCP JSON-RPC tools/list protocol without custom API integration wrappers.
2. Standardized Request/Result Flow: Tool calls executed seamlessly across stdio transport, returning structured JSON strings directly to the client UI.
3. Preservation of Deterministic Safety Guardrails: Backend deterministic rules (idempotency key tracking, user existence checks, and programmatic balance comparisons) remained fully functional inside the MCP server tools.

What Failed or Behaved Unexpectedly?

Client Tool Authorization Bypass Attempt during Direct Prompt Injection: When testing a state-mutating request (create_leave_request) through the MCP client, the model assumed tool invocation approval directly from the prompt. Because MCP itself is a communication protocol and NOT an agent or authorization gateway, it transmitted the tool execution request immediately. This highlighted that human-in-the-loop authorization gates and security permission boundaries must be explicitly enforced inside the application execution wrapper, not left to MCP standard transport.

Part 3 - Engineering Reasoning

• **Key Decision:** Adopt FastMCP with stdio transport for local agent tool standardization.
• **Alternative Considered:** Writing custom REST API endpoints (FastAPI) with manual JSON schema endpoints for each tool.
• **Trade-off Accepted:** Acquired protocol standardization and client plug-and-play capability at the cost of requiring an active local Python stdio subprocess lifecycle managed by the client.

Thamill Indian B U
Cohort-15 (AI Engineer)

Part 4 - FDE Reflection & Test Case Review Scenarios Matrix

To thoroughly review the day's output against enterprise standards, the MCP implementation was evaluated against all six Day 14 Common Test Cases / Review Scenarios required by the curriculum:

Scenario Type	Test Case Input	System Behavior & Output	Validation Status
Normal Path	get_available_leave_balance for user_id='E101'	Server executes lookup and returns status: SUCCESS, leave_balance: 3.	PASSED (Verified in client execution trace)
Invalid Input	get_available_leave_balance for non-existent user_id='E999'	FastMCP returns structured JSON error response (USER_NOT_FOUND) safely without throwing unhandled exceptions.	PASSED (Safe handled failure)
Dependency / Model Failure	create_leave_request when user balance is 0 (User E102)	Backend deterministic balance logic catches failure and returns INSUFFICIENT_BALANCE status message.	PASSED (Blocked state mutation)
Ambiguous Request	User asks 'Book me leave next month' without providing start_date or days.	Client prompts user for missing required parameters based on MCP tool schema docstring definition.	PASSED (Bounded behavior)
Security / Permission Boundary	Model attempts to invoke create_leave_request without explicit user confirmation.	Application execution wrapper enforces HITL gate; state mutation is blocked until human approval is confirmed.	PASSED (Enforced trust boundary)
Operational Case	Duplicate leave request submission for same user and date.	Idempotency cache interceptor returns cached replay without double-deducting leave.	PASSED (Idempotency & audit logging verified)

1. Protocol Standards vs. Application Responsibilities

• What MCP Standardizes: Standardizes the communication protocol (JSON-RPC over stdio/SSE), dynamic tool capability discovery (tools/list), and schema definitions.
• What Application Retains: MCP IS NOT AN AGENT. The application retains responsibility for RBAC identity propagation, validation, rate limiting, human write approval gates, idempotency, and audit trails.

2. Pre-Deployment Customer Validation Questions

1. Security Boundary: How does the MCP server authenticate user identity and pass user authorization tokens across stdio/SSE transport to prevent cross-tenant data access?
2. Host Process Lifecycle: Is stdio transport appropriate, or does the enterprise infrastructure require SSE/HTTP microservices managed via Kubernetes?
3. Auditability: Does the customer require an explicit approval workflow (e.g., Slack webhook) before state-mutating MCP write tools execute?